Optimizarea interogărilor SQL

Seminar 5

De ce contează optimizarea?

- Același rezultat se poate obține în moduri diferite (interogări diferite)
- Optimizatorul transformă interogarea primită într-un plan de execuție - dar lucrează cu datele primite
- Planul de execuție - optimizarea bazată pe cost

Optimizarea SQL

- Statistici: numărul de înregistrări, cardinalitatea, histograma – ceea ce știe BD
- Selectivitatea: câte înregistrări vor fi returnate de fiecare predicat
- Ordinea JOIN-urilor: filtrare imediată, join de rezultate mai puține
- Conceptul de bază: condițiile din WHERE aplicate cât mai devreme

Exemple:

Numărul de înregistrări

- Tabelul angajati are 100 de rânduri.
  Tabelul vanzari are 5.000.000 de rânduri.

- Baza de date știe că:
  - angajati este mic
  - vanzari este mare

- Consecință:
  - pentru angajati poate fi acceptabil un full scan
  - pentru vanzari va încerca metode mai eficiente, de exemplu index

Exemplu

Cardinalitatea

- În tabelul clienti, coloana oras are valorile: Bucuresti, Cluj, Iasi, Timisoara
- Sunt doar **4 valori distincte**, deci cardinalitate mică.
- Coloana email are valori aproape unice pentru fiecare client, deci cardinalitate mare.

- Consecință:
  - index pe email poate fi foarte util
  - index pe oras poate fi mai puțin util dacă multe rânduri au aceeași valoare

Exemplu

Tabelul **ORDERS** are 200.000 rânduri.
Tabelul **CUSTOMERS** are 1000 de înregistrări.

```
explain SELECT *
FROM orders o JOIN customers c ON c.id=o.customer_id
WHERE o.total > 100 AND c.country='France';
```

```
QUERY PLAN
------------------------------------------------------------------------
Hash Join  (cost=29.48..4037.24 rows=25518 width=41)
  Hash Cond: (o.customer_id = c.id)
  ->  Seq Scan on orders o  (cost=0.00..3582.00 rows=161509 width=14)
        Filter: (total > '100'::numeric)
  ->  Hash  (cost=27.50..27.50 rows=158 width=27)
        ->  Seq Scan on customers c  (cost=0.00..27.50 rows=158 width=27)
              Filter: ((country)::text = 'France'::text)
```

Exemplu

Avem:

- `facturi` = 3.000.000 rânduri
- `furnizori` = 20.000 rânduri
- `regiuni` = 50 rânduri

SQL:

```
SELECT *FROM facturi f
JOIN furnizori s ON f.furnizor_id = s.id
JOIN regiuni r ON s.regiune_id = r.id
WHERE r.nume = 'Europa'
```

Care e ordinea tabelelor din sql?

Exemplu

```
SELECT c.nume_client, o.id_comanda, p.nume_produs, f.nume_furnizor, a.oras, t.nume_tara

FROM clienti c JOIN comenzi o  ON c.id_client = o.id_client
      JOIN linii_comanda lc  ON o.id_comanda = lc.id_comanda
      JOIN produse p ON lc.id_produs = p.id_produs
      JOIN furnizori f  ON p.id_furnizor = f.id_furnizor
      JOIN adrese a ON c.id_adresa = a.id_adresa
      JOIN tari t ON a.id_tara = t.id_tara

WHERE
   c.status_client = 'ACTIV'
    AND o.status = 'FINALIZATA'
    AND o.valoare_totala > 10000
    AND p.stoc > 0
    AND p.categorie = 'Laptop'
    AND f.tara_origine = 'Germania'
    AND a.oras = 'Bucuresti'
     AND t.nume_tara = 'Romania';
```

Cum se rescrie optimizat sql-ul?

Solutie

```
SELECT c.nume_client, o.id_comanda, p.nume_produs, f.nume_furnizor, a.oras, t.nume_tara
FROM tari t
JOIN adrese a ON t.id_tara = a.id_tara AND a.oras = 'Bucuresti'
JOIN clienti c ON a.id_adresa = c.id_adresa AND c.status_client = 'ACTIV'
JOIN comenzi o ON c.id_client = o.id_client AND o.status = 'FINALIZATA'
AND o.valoare_totala > 10000
JOIN linii_comanda lc ON o.id_comanda = lc.id_comanda
JOIN produse p ON lc.id_produs = p.id_produs AND p.categorie = 'Laptop' AND p.stoc > 0
JOIN furnizori f ON p.id_furnizor = f.id_furnizor AND f.tara_origine = 'Germania'
WHERE t.nume_tara = 'Romania';
```

Interogări imbricate – JOINS - EXISTS

- (Sub) interogări corelate: executate pentru fiecare înregistrare - O(N) execuții
- (Sub) interogări necorelate: executate o singură data – O(1)
- JOIN: bazate pe relații/seturi - optimizatorul poate reordona liber accesul
- EXISTS vs IN vs JOIN - când este util fiecare operator

Interogări imbricate - slow

```
SELECT o.id, o.total
FROM Orders o
WHERE o.total > (
    SELECT AVG(total)
    FROM Orders o2
    WHERE o2.customer_id = o.customer_id
);
```

Interogări imbricate – soluția 1: JOIN

```
SELECT o.id, o.total, o.customer_id
FROM Orders o JOIN (
    SELECT customer_id, AVG(total) AS avg_total
    FROM Orders
    GROUP BY customer_id) AS customer_avg
    ON o.customer_id = customer_avg.customer_id
WHERE o.total > customer_avg.avg_total;
```

Interogări imbricate – soluția 2: WITH (CTE)

```
WITH customer_avg AS (
SELECT customer_id, AVG(total) AS avg_total
FROM Orders GROUP BY customer_id)
SELECT o.id, o.total, o.customer_id
FROM Orders o
    JOIN customer_avg ca ON o.customer_id = ca.customer_id WHERE o.total
> ca.avg_total;
```

Erori de evitat

- SELECT * - încarcă date neutilizate, irosire I/O
- Condiții OR -> UNION ALL
- DISTINCT pentru a corecta operațiuni JOIN greșite

OR vs UNION ALL

```
SELECT c.name, c.country, o.total
FROM customers c JOIN orders o ON c.id=o.customer_id
WHERE c.country='Japan' OR  o.total > 500;
```

```
Hash Join
 Hash Cond: (o.customer_id = c.id)
  Join Filter: (((c.country)::text = 'Japan'::text) OR (o.total > '500'::numeric))
  Rows Removed by Join Filter: 169924
  ->  Seq Scan on orders o
  ->  Hash
        ->  Seq Scan on customers c
```

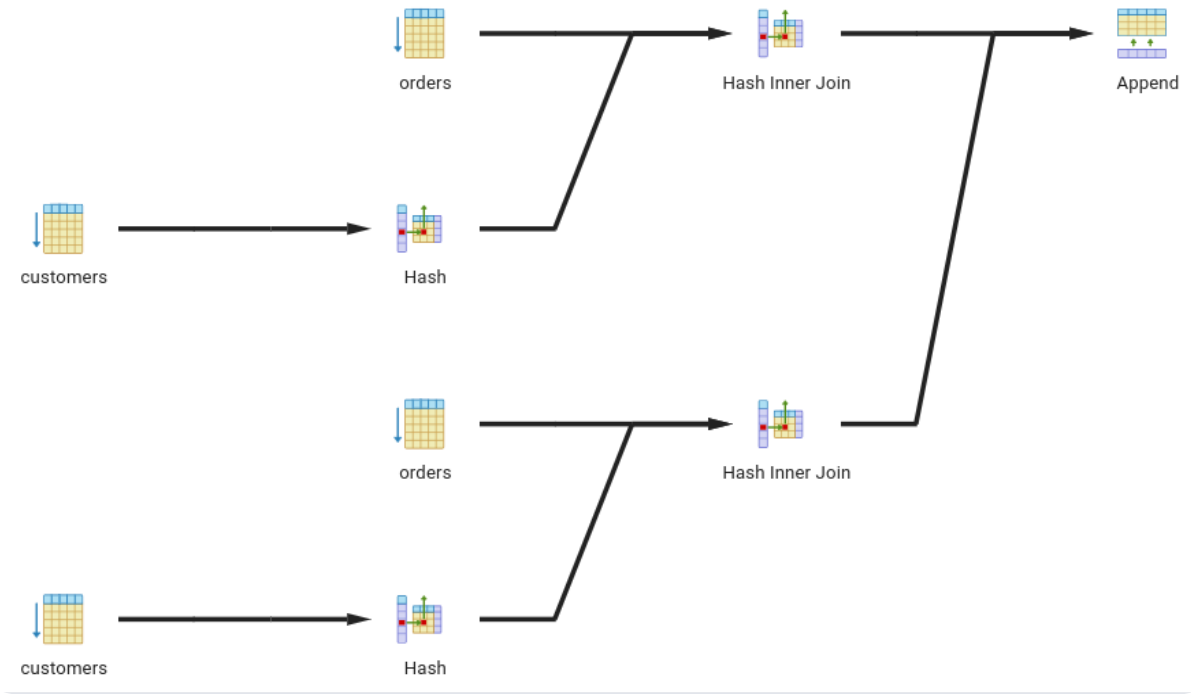
OR vs UNION ALL

```
SELECTc.name, c.country, o.total
FROM customers c join orders o ON c.id=o.customer_id
WHERE  c.country='Japan'
UNION ALL
SELECT c.name, c.country, o.total
FROM customers c join orders o ON c.id=o.customer_id
WHERE c.country<>'Japan' AND o.total > 500;
```

OR vs UNION ALL - Plan

```
Append
 ->  Hash Join
       Hash Cond: (o.customer_id = c.id)
       ->  Seq Scan on orders o
       ->  Hash
             ->  Seq Scan on customers c
                   Filter: ((country)::text = 'Japan'::text)
                   Rows Removed by Filter: 859
 ->  Hash Join
       Hash Cond: (o_1.customer_id = c_1.id)
       ->  Seq Scan on orders o_1
             Filter: (total > '500'::numeric)
             Rows Removed by Filter: 198072
       ->  Hash
             ->  Seq Scan on customers c_1
                   Filter: ((country)::text <> 'Japan'::text)
                   Rows Removed by Filter: 141
```

Planuri de execuție



Erori folosind JOIN

"Get the names of all Japanese customers who have placed at least one order."

```
SELECT c.name
FROM Customers c
JOIN Orders o ON c.id = o.customer_id
WHERE c.country = 'Japan';
```

Erori folosind JOIN - 2

"Get the names of all Japanese customers who have placed at least one order."

```
SELECT c.name
FROM Customers c
WHERE c.Country = 'Japan'
  AND EXISTS (
      SELECT 1
      FROM Orders o
      WHERE o.customer_id = c.id
  );
```

Exercitiu

- Authors: AuthorID, AuthorName, Nationality
- Books: BookID, AuthorID, Title, PageCount, Genre
- Loans: LoanID, BookID, MemberID, BranchID
- Members: MemberID, MemberName, MembershipType ('Premium', 'Basic')
- Branches: BranchID, BranchName, City, Status ('Active', 'Under Renovation')

Bibliotecta dorește un raport pentru a decide care autor merită promovat în luna următoare. Un autor se califică dacă:
Cărțile scrise de el au fost împrumutate de cel puțin 5 ori in total
Are cel puțin o carte cu peste 400 de pagini
Pentru fiecare autor calificat, raportul va afișa **numele, naționalitatea, numărul total de împrumuturi** pentru toate cărțile scrise și **numărul mediu de pagini**.
Se vor considera doar cărțile împrumutate în **agențiile active**. Se vor exclude autorii care **nu au cel puțin o carte** împrumutată.
Ordonați rezultatele în ordinea descrescătoare a numărului de împrumuturi.

Exercitiu

```
WITH AuthorMetrics AS (
   SELECT a.AuthorID, a.AuthorName, a.Nationality, COUNT(l.LoanID) AS TotalLoans,
      AVG(b.PageCount) AS AvgPageCount, MAX(b.PageCount) AS MaxPageCount
   FROM Authors a
      JOIN Books b ON a.AuthorID = b.AuthorID
      JOIN Loans l ON b.BookID = l.BookID
      JOIN Branches br ON l.BranchID = br.BranchID
   WHERE br.Status = 'Active'
   GROUP BY a.AuthorID, a.AuthorName, a.Nationality)
SELECT am.AuthorName, am.Nationality, am.TotalLoans, ROUND(am.AvgPageCount, 1) AS AvgPageCount
FROM AuthorMetrics am WHERE am.TotalLoans > 5
UNION
SELECT am.AuthorName, am.Nationality, am.TotalLoans, ROUND(am.AvgPageCount, 1) AS AvgPageCount
FROM AuthorMetrics am JOIN Books b ON am.AuthorID = b.AuthorID
WHERE b.PageCount > 400
ORDER BY TotalLoans DESC;
```

ORACLE - AWR Reports

SQL ordered by Executions

- %CPU - CPU Time as a percentage of Elapsed Time
- %IO - User I/O Time as a percentage of Elapsed Time
- Total Executions: 1,387,330
- Captured SQL account for 68.8% of Total

Executions	Rows Processed	Rows per Exec	Elapsed Time (s)	%CPU	%IO	SQL Id	SQL Module	SQL Text
150,959	150,992	1.00	2.72	14.9	0	65kfww9x61q88	JDBC Thin Client	select max(businessda0_.busine...
122,824	73,474	0.60	5.20	18.2	0	dgy4un1mj3yy1	JDBC Thin Client	select csdpartici0_.id as id2_...
106,049	106,059	1.00	2.46	14.2	0	gw58cbv5j4uur	JDBC Thin Client	select synthetica0_.id as id1_...
49,361	17,715	0.36	2.30	17.9	0	2r760xt4p3hak	JDBC Thin Client	select csdpartici0_.id as id2_...
43,098	43,096	1.00	2.24	20.5	0	5fsah9s5c79jc	JDBC Thin Client	select issue0_.id as id1_183_,...
39,924	39,927	1.00	4.92	28.1	0	bwfu41dyapp0v	JDBC Thin Client	select instructio0_.id as id2_...

AWR Reports – examplul 2

SQL ordered by Elapsed Time

- Resources reported for PL/SQL code includes the resources used by all SQL statements called by the code.
- % Total DB Time is the Elapsed Time of the SQL statement divided into the Total Database Time multiplied by 100
- %Total - Elapsed Time as a percentage of Total DB time
- %CPU - CPU Time as a percentage of Elapsed Time
- %IO - User I/O Time as a percentage of Elapsed Time
- Captured SQL account for 96.5% of Total DB Time (s): 18,726
- Captured PL/SQL account for 0.3% of Total DB Time (s): 18,726

Elapsed Time (s)	Executions	Elapsed Time per Exec (s)	%Total	%CPU	%IO	SQL Id	SQL Module	SQL Text
12,797.19	2,234	5.73	68.34	56.50	0.00	ab8xwyfahcjj1	JDBC Thin Client	select interfacem0_.tran_id as...
1,584.44	2,551	0.62	8.46	46.67	0.00	9uudc4bbawd28	JDBC Thin Client	select distinct csdtran0_.id a...
1,382.64	12,488	0.11	7.38	51.34	0.00	38cu033tpnvcg	JDBC Thin Client	select swifttrani0_.id as id2_...
384.71	1,399	0.27	2.05	46.44	0.00	400b3nfgjpk48	JDBC Thin Client	select csdtran0_.id as id1_123...
217.09	1,024	0.21	1.16	47.48	0.00	2uk5zw6zufbjg	JDBC Thin Client	select nsecorresp0_.id as id1_...
200.55	484	0.41	1.07	55.66	0.00	cfsf9vk9v3a3j	JDBC Thin Client	select inputs0_.tran_id as tra...
154.03	233	0.66	0.82	44.97	0.00	5rb09uqgbatrs	JDBC Thin Client	SELECT current_timestamp as c1...
145.50	13,751	0.01	0.78	47.82	0.00	3v17bkq4at7zq	JDBC Thin Client	select count(csdtran0_.id) as ...
130.03	12,488	0.01	0.69	51.82	0.00	cys45rkvs7bu1	JDBC Thin Client	select count(csdtran0_.id) as ...
126.07	256	0.49	0.67	48.00	0.00	3s9uu3vuaj5qx	JDBC Thin Client	select activitylo0_.id as id1_...

Back to SQL Statistics

Monitorizare - Investigație

1. Interogări ordonate după: timp de execuție, nr de execuții
2. Operații de I/O: pagini citite/scrise, buffere citite/scrise, Reads/sec
3. Verificare planuri de execuție la interogările care rulează încet
4. Verificare utilizare indecși

Resurse și referințe

- **SQL Server:**
  - https://learn.microsoft.com/en-us/sql/relational-databases/sql-server-transaction-locking-and-row-versioning-guide?view=sql-server-ver16
  - https://learn.microsoft.com/en-us/sql/relational-databases/extended-events/quick-start-extended-events-in-sql-server?view=sql-server-ver16
  - https://learn.microsoft.com/en-us/sql/relational-databases/system-stored-procedures/sp-lock-transact-sql?view=sql-server-ver16
  - https://learn.microsoft.com/en-us/sql/relational-databases/system-dynamic-management-views/sys-dm-tran-locks-transact-sql?view=sql-server-ver17
- **PostgreSQL:**
  - https://www.postgresql.org/docs/18/mvcc.html